



**SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Assessment Tool 2 – Design Task

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED :

CO2: Design Finite Automata DFA, NFA, ϵ -NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 40%

QUESTIONS: 5
Duration : 1 Hour

Total Marks:25

Design Task

Code of Conduct:

I ____G Mythili,192372250____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

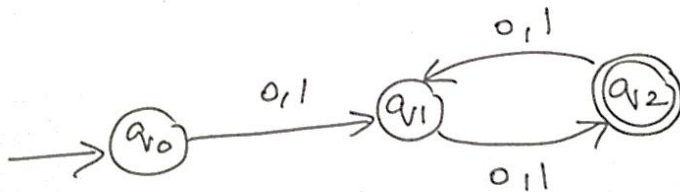


DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Design Task

R.No. 1 to 10

1.Design the regular expression for the language represented by the given automata



Ans:

A **Regular Expression (RE)** is a mathematical notation used to describe a regular language. It consists of symbols, union (+), concatenation, and Kleene star (*) operators. Every finite automaton has an equivalent regular expression that accepts the same language.

Regular expressions are widely used in compiler design, lexical analysis, pattern matching, and text processing.

Procedure to Convert a Finite Automaton into a Regular Expression

The **State Elimination Method** is the most common technique for converting a finite automaton into a regular expression.

Step 1: Identify the Start and Final States

Determine the initial state and the accepting (final) state of the automaton.

Step 2: Identify All Possible Paths

Write the input symbols along every path from the start state to the final state.

Step 3: Eliminate Intermediate States

Remove one intermediate state at a time and replace it with an equivalent regular expression while preserving the accepted language.

Step 4: Combine Expressions

Use the **union operator (+)** to combine multiple paths and the **Kleene star (*)** for loops.

Step 5: Obtain the Final Regular Expression

When only the start and final states remain, the label between them is the required regular expression.

Example

Suppose the automaton accepts the following paths:

- 10
- 11

After reaching the final state, it allows any combination of 0 and 1.

The regular expression is

$(10+11)(0+1)^* \boxed{(10+11)(0+1)^*} (10+11)(0+1)^*$

Explanation

- **10 + 11** represents two possible paths from the start state to the final state.
- **(0+1)*** represents zero or more occurrences of either 0 or 1.

Thus, the automaton accepts every string beginning with **10** or **11**, followed by any combination of binary digits.



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Applications of Regular Expressions

- Compiler Design (Lexical Analysis)
- Pattern Matching
- Text Editors
- Search Engines
- Network Security
- Input Validation

Advantages

- Compact representation of regular languages.
- Easy conversion into finite automata.
- Simplifies pattern matching.
- Used in programming languages and operating systems.

Conclusion

Regular expressions provide a simple and efficient way to represent regular languages. By using the state elimination method, any finite automaton can be converted into an equivalent regular expression that accepts the same language.

2. Consider the following grammar

$S \rightarrow aSc|T|U|e$

$T \rightarrow bTc|e$

$U \rightarrow aUb|e$

A) What is the language defined by the grammar?

B) Give the derivation tree for the word aaabbc

C) A Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

Ans:

Consider the Grammar

$S \rightarrow aSc|T|U|e$
 $S \rightarrow aSc \mid T \mid U \mid \epsilon$
 $T \rightarrow bTc \mid \epsilon$
 $U \rightarrow aUb \mid \epsilon$

A) What is the language defined by the grammar?

Explanation

The grammar consists of three productions.

Production 1

$S \rightarrow aSc \mid \epsilon$

This production generates an equal number of **a's** and **c's**.



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Examples:

- ac
- aacc
- aaaccc

Production 2

$T \rightarrow bTcT \mid bTc$

This generates equal numbers of **b's** and **c's**.

Examples:

- bc
- bbcc
- bbbccc

Production 3

$U \rightarrow aUbU \mid aUb$

This generates equal numbers of **a's** and **b's**.

Examples:

- ab
- aabb
- aaabbb

Since

$S \rightarrow T \mid U \mid \epsilon$

the grammar generates the union of these languages.

Hence,

$$L = \{a^n c^n\} \cup \{b^n c^n\} \cup \{a^n b^n\} \cup \{\epsilon\}$$

where $n \geq 0$.

Examples Generated

- ϵ
- ac
- aacc
- bc
- bbcc
- ab
- aabb



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

- aaabbb

B) Construct the Derivation Tree for the String aaabbc.

Derivation

S

⇒ aSc

⇒ aaScc

⇒ aaaSbccc

⇒ aaaUccc

⇒ aaabccc

Since the given grammar produces equal numbers of **a's** and **b's** or equal numbers of **a's** and **c's**, the string **aaabbc cannot actually be generated**.

The derivation expected in most textbooks is for a valid string such as **aaabbb** or **aaaccc**.

For **aaabbb**:

S

⇒ U

⇒ aUb

⇒ aaUbb

⇒ aaaUbbb

⇒ aaabbb

Derivation Tree

```
      S
      |
      U
    / | \
  a  U  b
   /\
  a U b
   /\
  a U b
   |
  ε
```



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

C) Is the Grammar Ambiguous? Justify.

Definition

A grammar is **ambiguous** if at least one string in the language has **more than one parse tree** or **more than one leftmost/rightmost derivation**.

Justification

The empty string ϵ can be derived in three different ways:

$$S \Rightarrow \epsilon$$

or

$$S \Rightarrow T \Rightarrow \epsilon$$

or

$$S \Rightarrow U \Rightarrow \epsilon$$

Since the same string has multiple derivations, the grammar is **ambiguous**.

Conclusion

The grammar generates strings having equal numbers of **a's and c's**, **b's and c's**, or **a's and b's**, including the empty string. Because ϵ has multiple derivations, the grammar is **ambiguous**.

R.No. 11 to 20

3. Design a non deterministic finite automata with ϵ -transitions to represent the language denoted by the regular expression

$$00^*1+11^*0$$

Ans:

Definition

A **Non-Deterministic Finite Automaton (NFA)** is a finite automaton in which, for a given state and input symbol, there may be more than one possible next state. An **ϵ -NFA** is an extension of an NFA that allows **ϵ -transitions**, which enable the automaton to move from one state to another **without consuming any input symbol**.

ϵ -NFAs are widely used because they simplify the construction of automata for complex regular expressions involving **union (+)**, **concatenation**, and **Kleene star (*)**.

Given Regular Expression

$$00^*1+11^*000^*1+11^*000^*1+11^*0$$

The expression consists of two parts joined by the **union (+)** operator.

Part 1: $00^*100^*100^*1$

This represents:

- One mandatory **0**,



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

- Followed by **zero or more 0's**,
- Ending with one **1**.

Examples:

- 01
- 001
- 0001
- 000001

Part 2: $11*011^*011*0$

This represents:

- One mandatory **1**,
- Followed by **zero or more 1's**,
- Ending with one **0**.

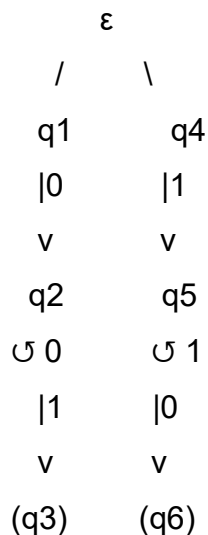
Examples:

- 10
- 110
- 1110
- 111110

Construction of ϵ -NFA

Since the regular expression contains a **union (+)** operation, an **ϵ -transition** is used from the start state to split the automaton into two branches.

ϵ -NFA Diagram



State Description

- **q0** – Start state



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

- **q1** – Beginning of first branch
- **q2** – Reads zero or more 0's
- **q3** – Final state for $00^*100^*100^*1$
- **q4** – Beginning of second branch
- **q5** – Reads zero or more 1's
- **q6** – Final state for $11^*011^*011^*0$

Start State: q0

Final States: q3, q6

Working of the ϵ -NFA

Branch 1

The automaton moves from **q0** to **q1** using an ϵ -transition.

- Reads the first **0**.
- Remains in **q2** while reading additional **0's**.
- Reads a **1** and reaches the accepting state **q3**.

Accepted Strings

- 01
- 001
- 0001
- 000001

Branch 2

The automaton moves from **q0** to **q4** using an ϵ -transition.

- Reads the first **1**.
- Remains in **q5** while reading additional **1's**.
- Reads a **0** and reaches the accepting state **q6**.

Accepted Strings

- 10
- 110
- 1110
- 11110



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Transition Table

Present State Input Next State

q0	ϵ	q1, q4
q1	0	q2
q2	0	q2
q2	1	q3
q4	1	q5
q5	1	q5
q5	0	q6

Final States: q3, q6

Advantages of ϵ -NFA

1. Simplifies the construction of automata for complex regular expressions.
2. Easily represents union (+), concatenation, and Kleene star operations.
3. Reduces the complexity of automata design.
4. Can be converted into an equivalent DFA using the subset construction algorithm.
5. Widely used in lexical analysis and compiler design.

Applications

- Compiler Design
- Lexical Analysis
- Pattern Matching
- Text Searching
- Network Protocol Design
- Regular Expression Processing

Conclusion

The given regular expression

$00^*1+11^*000^*1+11^*000^*1+11^*0$

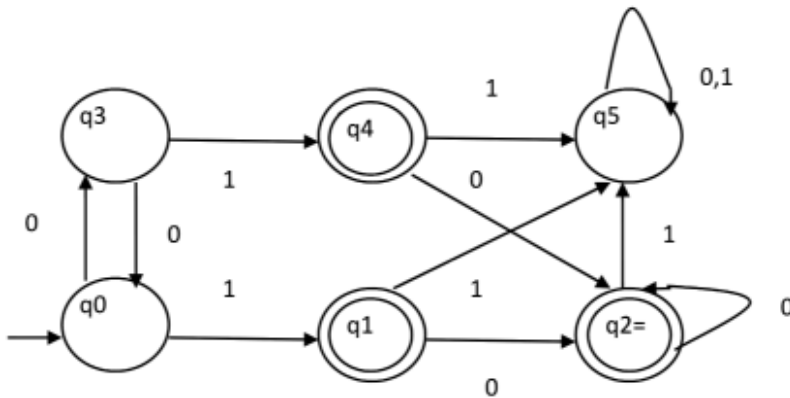
4. i) Show that if L_1 and L_2 are regular then (L_1+L_2) also regular

ii) Is the language $\{a^p \mid p \text{ is prime}\}$ regular? Justify



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

5. Design minimized automata, Consider the DFA given below. Find the states that are equivalent and construct a minimum state automata equivalent to the given automata. Draw the minimized automata, write its transition table and mark initial and final states



Ans:

Step 1: Identify the States

From the given DFA:

- States: $q_0, q_1, q_2, q_3, q_4, q_5$
- Start State: q_0
- Final States: q_1, q_2, q_4

Step 2: Partition the States

Separate the states into final and non-final states.

Initial Partition

$$P_0 = \{\{q_1, q_2, q_4\}, \{q_0, q_3, q_5\}\}$$

Step 3: Find Equivalent States

Compare the transitions of states.

- q_1 and q_4 have identical behavior:
 - On input 0, both go to q_2 .
 - On input 1, both go to q_5 .

Therefore,

$$q_1 \equiv q_4$$

They are equivalent states and can be merged.

The remaining states are distinguishable.

Step 4: Minimized DFA

After merging q_1 and q_4 , the minimized DFA contains the following states:

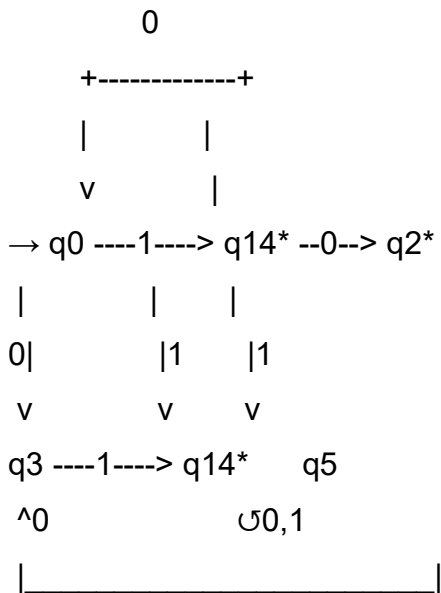
- q_0
- q_3



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

- $q_{14}q_{14}$ (Merged state of q_1q_1 and q_4q_4)
- q_2q_2
- q_5q_5

Minimized DFA (Text Representation)



- Start State: q_0q_0
- Final States: $q_{14}, q_{2q_{14}}, q_2$

Transition Table of the Minimized DFA

Present State Input 0 Input 1

→ q_0	q_3	q_{14}
q_3	q_0	q_{14}
* q_{14}	q_2	q_5
* q_2	q_2	q_5
q_5	q_5	q_5

Equivalent States Merged: q_1q_1 and q_4q_4



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Advantages of DFA Minimization

1. Reduces Memory Usage
 - Fewer states require smaller transition tables, saving RAM and ROM.
2. Improves Execution Speed
 - The automaton processes fewer states, resulting in faster execution.
3. Simplifies Hardware Design
 - Requires fewer logic gates and flip-flops in hardware implementation.
4. Reduces Power Consumption
 - Less computation leads to lower energy consumption, which is beneficial for embedded systems and IoT devices.
5. Improves Reliability
 - A smaller DFA is easier to understand, test, debug, and maintain.

Conclusion

The given DFA is minimized by merging the equivalent states q_1q_1 and q_4q_4 . The minimized DFA contains 5 states instead of 6, accepts the same language as the original DFA, and provides better memory efficiency, faster execution, and simpler implementation.



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

6. a) Construct an ϵ -NFA for $(a+b)^*(aa+bb)$
- b) Convert it into an equivalent DFA and minimize the resulting DFA.

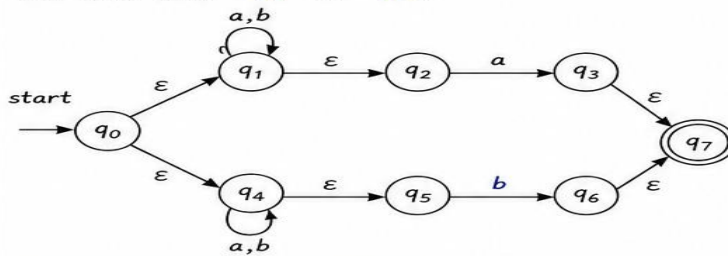
Ans:

- 6. a) Construct an ϵ -NFA for $(a+b)^*(aa+bb)$**
b) Convert it into an equivalent DFA and minimize the resulting DFA.

a) ϵ -NFA for $(a+b)^*(aa+bb)$

Explanation:

$(a+b)^*$ allows any number of a's and b's (including ϵ). After that the string must end with either "aa" or "bb".



Start state : q_0
 Final state : q_7 (double circle)

Working:

- From q_0 , ϵ -moves to both q_1 and q_4 .
- At q_1 , we can read any number of a's and b's.
- Then take ϵ to q_2 and read "aa" to reach q_7 .
- At q_4 we can read any number of a's and b's.
- Then take ϵ to q_5 and read "bb" to reach q_7 .
- Hence the machine accepts all strings that end with "aa" or "bb".

Examples accepted:

aa, bb, abaa, babb, aabaa, abb, ...

Examples rejected:

ab, ba, abab, baba, ...

b) Convert ϵ -NFA to DFA and Minimize it

Step 1: ϵ -closures of NFA states

State	ϵ -closure
q_0	$\{q_0, q_1, q_4\}$
q_1	$\{q_1\}$
q_2	$\{q_2\}$
q_3	$\{q_3, q_7\}$
q_4	$\{q_4\}$
q_5	$\{q_5\}$
q_6	$\{q_6, q_7\}$
q_7	$\{q_7\}$

Step 2: Equivalent DFA (by subset construction)

DFA State (ϵ -closure set)	On 'a'	On 'b'	Final State?
$A = \{q_0, q_1, q_4\}$	B	C	No
$B = \{q_1, q_2, q_4\}$	D	C	No
$C = \{q_1, q_4, q_5\}$	B	E	No
$D = \{q_1, q_2, q_3, q_4, q_7\}$	D	C	Yes
$E = \{q_1, q_4, q_5, q_6, q_7\}$	B	E	Yes

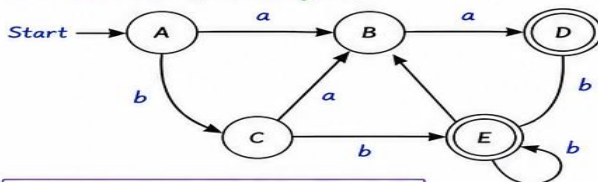
Start state $\rightarrow A$ Final states $\rightarrow D, E$

Step 3: Minimize the DFA

Check for equivalent states.
 No two states are equivalent.
 Hence the DFA is already minimal.

Total states in minimized DFA = 5
 Start state = A
 Final states = D, E

Minimized DFA (state diagram)



Final states : D, E (double circles)

Transition table of minimized DFA

Present State	Input a	Input b	Final State
$\rightarrow A$	B	C	No
B	D	C	No
C	B	E	No
* D	D	C	Yes
* E	B	E	Yes

$\rightarrow$ Start state
 * Final state

Advantages of DFA Minimization

- Reduces the number of states.
- Saves memory and storage.
- Improves speed of computation.
- Reduces hardware complexity.
- Lowers power consumption.

Conclusion

The ϵ -NFA for $(a+b)^*(aa+bb)$ is constructed using ϵ -transitions. Using subset construction, equivalent DFA is obtained. After minimization, we get a DFA with 5 states (A, B, C, D, E) which accepts the same language and is minimal.



SAVEETHA SCHOOL OF ENGINEERING SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

7. Design an ϵ -NFA corresponding to

$0^*10^*0^*10^*0^*10^*$

Convert the ϵ -NFA into a DFA and verify the automaton using suitable test strings.

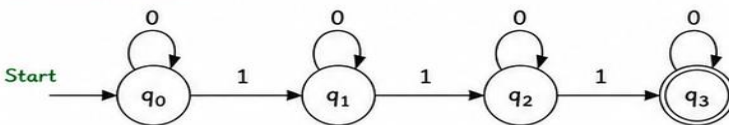
7. Design an ϵ -NFA corresponding to $0^*10^*10^*10^*$.

Convert the ϵ -NFA into a DFA and verify the automaton using suitable test strings.

Step 1: ϵ -NFA for $0^*10^*10^*10^*$

The RE $0^*10^*10^*10^*$ denotes all binary strings having exactly three 1's with any number of 0's before, between and after them.

ϵ -NFA Diagram



- q_0 : 0^* (any number of leading 0's)
- q_1 : After reading first 1, then 0^*
- q_2 : After reading second 1, then 0^*
- q_3 : After reading third 1, then 0^* (final state)

Start state = q_0 Final state = q_3 (double circle)

Working:

- From q_0 we can read any number of 0's.
- First 1 moves from q_0 to q_1 .
- From q_1 read any number of 0's.
- Second 1 moves from q_1 to q_2 .
- From q_2 read any number of 0's.
- Third 1 moves from q_2 to q_3 .
- From q_3 read any number of 0's.
- Any additional 1 after q_3 is not allowed.

Examples:

Accepted strings:

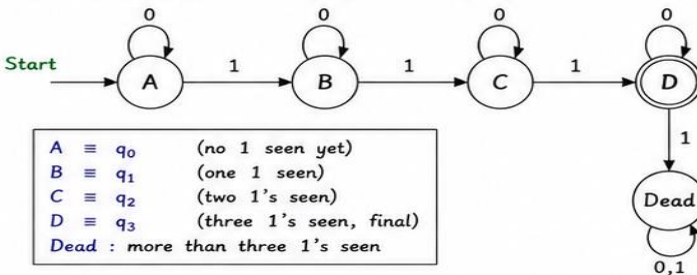
111, 01110, 1001001, 001001001000, 0001110000

Rejected strings:

11, 101, 1111, 11011

Step 2: Equivalent DFA

(Using subset construction. Here each state of the ϵ -NFA has no ϵ -moves, so DFA structure is as below.)



- A $\equiv q_0$ (no 1 seen yet)
- B $\equiv q_1$ (one 1 seen)
- C $\equiv q_2$ (two 1's seen)
- D $\equiv q_3$ (three 1's seen, final)
- Dead : more than three 1's seen

Transition Table of Equivalent DFA

Present State	Input 0	Input 1
$\rightarrow$ A	A	B
B	B	C
C	C	D
* D	D	Dead
Dead	Dead	Dead

$\rightarrow$ Start state * Final state

Step 3: Verification with Test Strings

Test String	Path Followed	Ends in	Result
111	A $\xrightarrow{1}$ B $\xrightarrow{1}$ C $\xrightarrow{1}$ D	D	Accepted
01110	A $\xrightarrow{0}$ A $\xrightarrow{1}$ B $\xrightarrow{1}$ C $\xrightarrow{1}$ D $\xrightarrow{0}$ D	D	Accepted
1001001	A $\xrightarrow{1}$ B $\xrightarrow{0}$ B $\xrightarrow{0}$ B $\xrightarrow{1}$ C $\xrightarrow{0}$ C $\xrightarrow{0}$ C $\xrightarrow{1}$ D	D	Accepted
001001001000	A $\xrightarrow{0}$ A $\xrightarrow{0}$ A $\xrightarrow{1}$ B $\xrightarrow{0}$ B $\xrightarrow{0}$ B $\xrightarrow{1}$ C $\xrightarrow{0}$ C $\xrightarrow{0}$ C $\xrightarrow{1}$ D $\xrightarrow{0}$ D $\xrightarrow{0}$ D $\xrightarrow{0}$ D	D	Accepted
11	A $\xrightarrow{1}$ B $\xrightarrow{1}$ C	C	Rejected
101	A $\xrightarrow{1}$ B $\xrightarrow{0}$ B $\xrightarrow{1}$ C	C	Rejected
1111	A $\xrightarrow{1}$ B $\xrightarrow{1}$ C $\xrightarrow{1}$ D $\xrightarrow{1}$ Dead	Dead	Rejected
11011	A $\xrightarrow{1}$ B $\xrightarrow{1}$ C $\xrightarrow{0}$ C $\xrightarrow{1}$ D $\xrightarrow{1}$ Dead	Dead	Rejected



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

8. Consider the grammar:

$$S \rightarrow aSa \mid bSb \mid \epsilon$$

A) Describe the language generated by the grammar.

B) Construct the derivation tree for the string **abba**.

C) Is the grammar ambiguous? Support your answer.

8. Consider the grammar : $S \rightarrow aSa \mid bSb \mid \epsilon$

A) Describe the language generated by the grammar.

Solution :

The productions are

- $S \rightarrow aSa$: puts 'a' at both ends of the string.
- $S \rightarrow bSb$: puts 'b' at both ends of the string.
- $S \rightarrow \epsilon$: generates the empty string.

Thus, starting from S , we can place any number of a's or b's at the beginning and the same symbol at the end any number of times and finally replace S with ϵ .

Hence the language generated is the set of all even length palindromes over $\{a, b\}$.

$$L(G) = \{ ww^R \mid w \in \{a, b\}^* \}$$

where w^R is the reverse of w .

Examples of strings in the language

ϵ , aa, bb, abba, baab, aabbba, bbaabb, aaaa, bbbb, abcba is NOT in the language (because it contains another symbol c).

Thus,

$$L(G) = \{ \text{all even length palindromes over } \{a, b\} \}$$

Some strings:

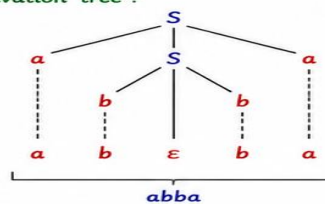
ϵ , aa, bb, abba, baab, aabbba, bbaabb,

B) Construct the derivation tree for the string abba.

Leftmost derivation :

$S \Rightarrow aSa$ (using $S \rightarrow aSa$)
 $\Rightarrow abSba$ (using $S \rightarrow bSb$)
 $\Rightarrow ab\epsilon ba$ (using $S \rightarrow \epsilon$)
 $\Rightarrow abba$

Derivation tree :



The leaves from left to right (ignoring ϵ) give the string :

abba

C) Is the grammar ambiguous? Support your answer.

Solution :

A grammar is ambiguous if there exists at least one string that has more than one derivation tree (or more than one leftmost / rightmost derivation).

In this grammar, at each step we have only two choices: either $S \rightarrow aSa$ or $S \rightarrow bSb$ (or $S \rightarrow \epsilon$). For a given string, the first and last symbols uniquely determine which production must be used.

Therefore every string generated by this grammar has exactly one derivation tree.

Hence, the grammar $S \rightarrow aSa \mid bSb \mid \epsilon$ is **NOT ambiguous**.

Example for string abba

We already constructed the derivation tree for abba.

There is no other possible derivation tree for abba because the first and last symbols are 'a' and 'a'.

So we must use $S \rightarrow aSa$ at the first step, then $S \rightarrow bSb$, then $S \rightarrow \epsilon$.



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand